Data Structures — Core Exam Points: Final Review Handout (Modern C++ Edition)

> **Revision note**: this document is a comprehensive reconstruction based on the original syllabus. It fixes the formula loss, garbled sentence breaks, and outdated syntax of the original draft; all core code has been stripped of C-style memory management and standardized to modern C++ object-oriented, reference-passing style.

Introduction: Pitfall-Avoidance Guide for the Final Exam (annotated by Akiro)

1. **The rift in language standards**: when answering on a written final-exam paper, **handwritten code must always use raw pointers and references (`&`)** — never write `std::shared_ptr` or `auto` on the answer sheet, as old-school professors will most likely mark it wrong. Dynamic memory allocation, however, must use `new / delete`, dropping `malloc / free`.
2. **The index trap (a life-or-death line on the answer sheet)**: in Yan Weimin's textbook, the valid elements of sequential lists, strings, and arrays default to **starting at index 1** (`S[0]` is often used to store the length or a sentinel). **But the modern C++ convention starts at 0**. In this handout, **all code strictly follows the 0-based standard**, yet at every core formula I have annotated the "textbook indexing convention" for you. Be sure to align with your instructor's PPT convention before entering the exam room!

Chapter 1 Introduction

1.1 What Data Structures Studies

For program-design problems involving **non-numerical computation**, it studies the objects the computer operates on, together with their relationships and operations.

Plaintext

```
What data structures covers:
 ├─ Logical structure
 │   ├─ Linear structure (linear lists, stacks, queues, strings, arrays)
 │   └─ Nonlinear structure (tree structure, graph structure)
 ├─ Physical (storage) structure
 │   ├─ Sequential storage structure
 │   ├─ Linked storage structure
 │   ├─ Indexed storage structure
 │   └─ Hashed (hash) storage structure
 └─ Data operations: insert, delete, modify, search, sort
```

1.2 Basic Concepts

- **Data**: the set of all symbols that a computer can recognize, store, and process.

- **Data element**: the **basic unit** of data, with a complete and definite practical meaning.
- **Data object**: a collection of data elements with the same properties; a subset of data.
- **Data structure**: a collection of data elements that have one or more specific relationships among one another; expressed as the mathematical model:

$$Data_Structure = (D, R)$$

(where D is a finite set of data elements and R is a finite set of relations on D)

- **Data type**: the general term for a **set of values** and the **set of operations** defined on those values.
- **Abstract data type (ADT, Abstract Data Type)**: a mathematical model defined by the user together with a set of operations defined on that model; it is built from basic data types.

1.3 Algorithms and Analysis

- **The five basic characteristics of an algorithm**: input, output, **finiteness**, **definiteness**, **feasibility**.
- **The four requirements of algorithm design**: ① correctness, ② readability, ③ robustness, ④ high efficiency and low storage demand.
- **Metrics for evaluating algorithm efficiency**:
  - **Time complexity**: estimated with the frequency function of execution counts $T(n) = O(f(n))$.
  - **Space complexity**: the size of the auxiliary space occupied while the algorithm runs, $S(n) = O(g(n))$.
  - **Stability**: *(the concept is flagged here; it is a key sorting topic examined in Chapter 9)*.

Chapter 2 Linear Lists (Including Arrays and Sparse Matrices)

2.1 Logical and Physical Structure of Linear Lists

Logical definition: a finite sequence $(a_1, a_2, \ldots, a_n)$ composed of n ($n \geq 0$) data elements. When $n = 0$, it is called an **empty list**.

Storage structure	Core physical property	Space density	Access mode	Typical use
Sequential list	logically adjacent, **absolutely adjacent in physical memory**	1.0 (very high)	**random access** $O(1)$	many lookups, few changes, fixed length
Linked list	logically adjacent, physical memory placement arbitrary	< 1.0	**sequential access** $O(n)$	frequent insert/delete, dynamic length

2.2 Efficiency of Core Sequential-List Operations

- **Modify**: core statement `V[i] = x;` ; time complexity $O(1)$.
- **Insert**: inserting before position i moves on average $\frac{n}{2}$ elements; time complexity $O(n)$.

(valid position condition: $1 \leq i \leq n+1$)

- **Delete**: deleting the i-th element moves on average $\frac{n-1}{2}$ elements; time complexity $O(n)$.

2.3 Singly Linked List (Modern C++ Code in Practice)

Exam Problem 1: Build a singly linked list of the 26 English letters

C++

```
#include <iostream>

struct ListNode {
    char data;
    ListNode* next;
    // C++ constructor initializer list — a bonus point on the answer sheet
    ListNode(char val = 0, ListNode* p = nullptr) : data(val), next(p) {}
};

// Standard form: build with tail insertion using a head node
ListNode* buildAlphaList() {
    ListNode* head = new ListNode(); // head node (stores no valid character)
    ListNode* tail = head;

    for (int i = 0; i < 26; ++i) {
        tail->next = new ListNode('a' + i);
        tail = tail->next;
    }
    return head;
}

void printList(ListNode* head) {
    ListNode* p = head->next; // skip the head node
    while (p != nullptr) {
        std::cout << p->data << " ";
        p = p->next;
    }
    std::cout << std::endl;
}
```

Exam Problem 2: Read the i-th element of a singly linked list

C++

```
ListNode* getElem(ListNode* head, int i) {
    if (i < 1) return nullptr;
    ListNode* p = head->next;
    int j = 1;

    while (p != nullptr && j < i) {
        p = p->next;
        ++j;
    }
    return p; // if i exceeds the list length, nullptr is returned naturally
}
```

Exam Problem 3: Core atomic logic for insertion and deletion in a singly linked list

> ⚠️ **Golden rule for the answer sheet**: for linked-list insertion, **attach first, link second**; for deletion, **re-link first, destroy second**. Never reverse the order!

- **Post-insertion in a singly linked list (insert new node `s` after node `p`)**:

  C++

  ```
  s->next = p->next;
  p->next = s;
  ```

- **Deletion in a singly linked list (given predecessor node `p`, delete its successor `q`)**:

  C++

  ```
  ListNode* q = p->next;
  p->next = q->next;
  delete q; // C++ frees memory with delete
  ```

Exam Problem 4: Core logic of the doubly linked list (Doubly Linked List)

Let pointer `p` point to the i-th element of the list; to insert new node `s` **before** `p`:

C++

```
s->prior = p->prior;
p->prior->next = s;
s->next = p;
p->prior = s;
```

2.4 Memory Addressing Formulas for Arrays (Mandatory Calculation Problems)

Let a two-dimensional array be $A[c_1 \dots d_1][c_2 \dots d_2]$ with base address $LOC(a_{c_1,c_2})$, each element occupying L bytes:

1. **Row-major storage address formula**:

$$LOC(a_{i,j}) = LOC(a_{c_1,c_2}) + \big[(i - c_1) \times (d_2 - c_2 + 1) + (j - c_2)\big] \times L$$

2. **Column-major storage address formula**:

$$LOC(a_{i,j}) = LOC(a_{c_1,c_2}) + \big[(j - c_2) \times (d_1 - c_1 + 1) + (i - c_1)\big] \times L$$

2.5 Sparse Matrices

A matrix is sparse when the number of nonzero elements is $< 5\%$.

- **Compressed storage options**: ① triplet table `(i, j, value)`; ② orthogonal (cross) linked list (each node has 5 fields: `row`, `col`, `val`, `down`, `right`).
- **Cost**: random access is lost.

Chapter 3 Stacks and Queues

3.1 The Stack — a Linear List with Restricted Operations (LIFO)

Insertion and deletion are allowed at only one end (the top of the stack, `top`).

> **Mainstream convention (initial top convention)**:
>
> - If initialized with `top = 0`: push is `S[top++] = e`; pop is `e = S[--top]`.
> - If initialized with `top = -1`: push is `S[++top] = e`; pop is `e = S[top--]`. *(this handout uses the first)*

Sequential Stack: Classic C++ Class Encapsulation

C++

```
template <typename T>
class SeqStack {
private:
    T* data;
    int top;
    int max_size;
public:
    SeqStack(int size = 100) : max_size(size), top(0) {
        data = new T[max_size];
    }

    bool push(const T& e) {
        if (top >= max_size) return false; // overflow
        data[top++] = e;
        return true;
    }

    bool pop(T& e) {
        if (top == 0) return false; // underflow
        e = data[--top];
        return true;
    }

    ~SeqStack() { delete[] data; }
};
```

- **Core engineering uses of the stack**: ① maintaining the stack frames underlying function calls; ② **recursive computation**; ③ saving and restoring the scene (context).

3.2 The Circular Queue — FIFO

Core pain point: in an ordinary sequential queue, as elements are continually enqueued and dequeued the pointers only increase and never decrease, which leads to "false overflow".

Solution: bend the one-dimensional array into a ring conceptually.

The Four Decision Formulas of the Circular Queue (the one-slot-sacrifice method)

Let the maximum capacity of the queue be `MAXSIZE`:

1. **Empty condition**: `front == rear`
2. **Full condition**: `(rear + 1) % MAXSIZE == front`
3. **Number of elements in the queue**:

$$L = (rear - front + MAXSIZE)\%MAXSIZE$$

4. **Pointer advancement**: enqueue advances `rear = (rear + 1) % MAXSIZE`; dequeue advances `front = (front + 1) % MAXSIZE`.

C++

```
template <typename T>
class CircularQueue {
private:
    T* base;
    int front;
    int rear;
    int max_size;
public:
    CircularQueue(int size = 100) : max_size(size), front(0), rear(0) {
        base = new T[max_size];
    }

    bool enQueue(const T& e) {
        if ((rear + 1) % max_size == front) return false; // queue full
        base[rear] = e;
        rear = (rear + 1) % max_size;
        return true;
    }

    bool deQueue(T& e) {
        if (front == rear) return false; // queue empty
        e = base[front];
        front = (front + 1) % max_size;
        return true;
    }
};
```

Chapter 4 Strings

- **Concept distinctions**:
  - **Empty (null) string**: a string of length 0;
  - **Blank string**: a string containing one or more **space** characters.
- **Core theorem**: the empty string is a substring of any string; any string is a substring of itself; substrings other than the string itself are called **proper substrings**.

4.1 The Brute-Force (BF) Pattern-Matching Algorithm

C++

```
#include <string>

// standard 0-based index implementation
int Index_BF(const std::string& S, const std::string& T, int pos = 0) {
    int i = pos;
    int j = 0;
    int s_len = S.length(), t_len = T.length();

    while (i < s_len && j < t_len) {
        if (S[i] == T[j]) {
            ++i; ++j;
        } else {
            i = i - j + 1; // the text pointer backtracks to just after the starting position of
the previous round
            j = 0;         // reset the pattern pointer
        }
    }
    return (j == t_len) ? (i - t_len) : -1;
}
```

> ⚠️ **Exam-point annotation**: the `i = i - j + 2` in the original document is a formula specific to the 1-based indexing of Yan Weimin's textbook edition, where `S[0]` stores the string length. When writing code on the final exam, always choose the backtracking formula according to the string definition given on the paper!

Chapter 6 Trees and Binary Trees

6.1 The Five Must-Know Properties of Binary Trees

1. A binary tree has at most 2^{i-1} nodes on level i ($i \geq 1$).
2. A binary tree of depth k has at most $2^k - 1$ nodes ($k \geq 1$).
3. **Leaf-node count formula (a mandatory proof and fill-in-the-blank)**: for any binary tree, if the number of nodes of degree 2 is n_2, the number of leaves n_0 is given by:

$$n_0 = n_2 + 1$$

(derivation: the total number of nodes is $n = n_0 + n_1 + n_2$; the total number of edges is $B = n - 1 = n_1 + 2n_2$; solving the two equations together and subtracting yields the result)

4. The depth of a **complete binary tree** with n nodes is $\lfloor \log_2 n \rfloor + 1$.
5. **Numbering property of the complete binary tree** (numbered from 1, top to bottom, left to right):
    - left child's number: $2i$
    - right child's number: $2i + 1$
    - parent node's number: $\lfloor i/2 \rfloor$

6.2 Binary-Tree Traversal and Forest Conversion

- **Traversal orders**:
  - Preorder traversal (DLR): root $\rightarrow$ left $\rightarrow$ right
  - Inorder traversal (LDR): left $\rightarrow$ root $\rightarrow$ right
  - Postorder traversal (LRD): left $\rightarrow$ right $\rightarrow$ root
- **Rule for uniquely reconstructing a binary tree**: **(preorder or postorder) + inorder** $\Rightarrow$ the binary tree can be uniquely determined. *Preorder and postorder alone cannot determine it!*

Conversion Rules between Tree/Forest $\longleftrightarrow$ Binary Tree (Left-Child, Right-Sibling)

- **Connect**: link all sibling nodes together.
- **Erase lines**: for each node in the tree, keep only its line to the eldest child (left child) and delete the lines to the other children.
- **Rotate**: rotate clockwise by 45°.

> *Note: in the binary tree obtained from a tree, the **root node necessarily has no right child**.*

6.3 Huffman Trees

- **Definition**: the binary tree with the shortest weighted path length (WPL) (the optimal binary tree).
- **Construction principle**: **larger weights sit closer to the root, smaller weights farther from the root**.
- **Weighted path length formula**:

$$WPL = \sum_{k=1}^{n} w_k l_k$$

- **Huffman coding**: a prefix code of unequal lengths; label left branches `0` and right branches `1`, achieving minimal redundancy over the full communication path.

Chapter 7 Graphs

$$G = (V, E)$$

7.1 Comparing Graph Storage Structures

1. **Adjacency matrix**:
   - Stores edges in a two-dimensional array `A[n][n]`.
   - Space complexity $O(n^2)$; **suited to dense graphs**.
   - In a directed graph: the number of nonzeros in the i-th **row** = the **out-degree** of vertex v_i; the number of nonzeros in the i-th **column** = the **in-degree**.
2. **Adjacency list**:
   - Stores the head node of each vertex sequentially, each head node carrying a singly linked list of its adjacent edges.
   - Space complexity $O(n + e)$; **suited to sparse graphs**.

7.2 Core Engineering Calculation Problems on Graphs (Final-Exam Focus)

Problem 1: Minimum Spanning Tree (MST, Minimum Spanning Tree)

A minimally connected subgraph that contains all n vertices but **has only $n-1$ edges**.

Algorithm	Core strategy	Merge target	Time complexity	Applicable graphs
Prim	"advance step by step", constantly seeking the **vertex** nearest the current tree	**merges vertices**	$O(n^2)$	**dense graphs**
Kruskal	"global optimum", picking the minimum-weight **edge** that forms no cycle	**merges edges**	$O(e\log_2 e)$	**sparse graphs**

Problem 2: Shortest Paths

1. **Single-source shortest path**: **Dijkstra's algorithm** (greedy idea; produced in increasing order of path length; **edge weights cannot be negative**).
2. **All-pairs shortest paths**: **Floyd's algorithm** (dynamic programming; triple loop, $O(n^3)$).

Chapter 8 Searching

- **Core evaluation metric**: average search length $ASL=\sum P_iC_i$.

8.1 Binary Search

- **Prerequisites**: ① sequential storage structure; ② records ordered by key.

C++

```
int BinarySearch(const int arr[], int n, int key) {
    int low = 0, high = n - 1;

    while (low <= high) {
        // engineering bonus: this form prevents integer overflow in low+high
        int mid = low + (high - low) / 2;

        if (arr[mid] == key) return mid;
        else if (arr[mid] > key) high = mid - 1;
        else low = mid + 1;
    }
    return -1;
}
```

- **Depth of the binary-search decision tree**: $\lfloor\log_2 n\rfloor+1$.
- **Approximate formula for ASL on success**:

$$ASL\approx\log_2(n+1)-1$$

8.2 Binary Search Trees (BST) and Balanced Binary Trees (AVL)

- **BST property**: the left subtree is all smaller, the right subtree all larger. **An inorder traversal of a BST absolutely yields an increasing ordered sequence**.
- **AVL tree definition**: the absolute value of the difference in depth between the left and right subtrees of any node is ≤ 1.
  - **Balance factor (BF)**: $BF = Depth_{left} - Depth_{right} \in \{-1, 0, 1\}$.
- **The four AVL rebalancing rotations (hand-drawn exam point)**:
  1. **LL case**: insertion in the left child's left subtree $\rightarrow$ **single right rotation** (about B as the pivot).
  2. **RR case**: insertion in the right child's right subtree $\rightarrow$ **single left rotation** (about B as the pivot).
  3. **LR case**: insertion in the left child's right subtree $\rightarrow$ **left rotation then right rotation**.
  4. **RL case**: insertion in the right child's left subtree $\rightarrow$ **right rotation then left rotation**.

Chapter 9 Internal Sorting

9.1 The Ultimate Side-by-Side Comparison Table of Exam Points

Sorting category	Algorithm	Average time	Worst-case time	Auxiliary space	Stability	Core logical characteristic
Insertion family	straight insertion sort	$O(n^2)$	$O(n^2)$	$O(1)$	**stable**	ordered as it inserts — the card-dealing logic
	binary insertion sort	$O(n^2)$	$O(n^2)$	$O(1)$	**stable**	finds faster — $O(n \log_2 n)$ comparisons but many more moves
	Shell sort	$O(n^{1.3})$	$O(n^2)$	$O(1)$	**unstable**	shrinking increments, grouped jump insertion
Exchange family	bubble sort	$O(n^2)$	$O(n^2)$	$O(1)$	**stable**	squeezes one largest bubble to the end each pass
	quicksort	$O(n \log_2 n)$	$O(n^2)$	$O(\log_2 n)$	**unstable**	find the pivot, Partition, divide and conquer
Selection family	simple selection sort	$O(n^2)$	$O(n^2)$	$O(1)$	**unstable**	pick the remaining minimum and swap it to the front
	heapsort	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	**unstable**	selection built on a complete binary tree
Merge family	two-way merge sort	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	**stable**	divide, conquer and merge; trades space for time

> 🔥 **Major errata alert on exam points**:

> The original syllabus never spelled out the **stability of simple selection sort**. Simple selection sort is **unstable**!
>
> *Counterexample proof*: for the sequence `[5, 5*, 2]`, the first pass finds the minimum element `2` and swaps it with the leading `5`, giving `[2, 5*, 5]` — the relative order of the two `5`s is instantly reversed.

9.2 Quicksort's Core Divide-and-Conquer Code (Partition)

C++

```
int partition(int arr[], int low, int high) {
    int pivot = arr[low]; // take the first element as the pivot

    while (low < high) {
        while (low < high && arr[high] >= pivot) --high;
        arr[low] = arr[high]; // throw anything smaller than the pivot to the left

        while (low < high && arr[low] <= pivot) ++low;
        arr[high] = arr[low]; // throw anything larger than the pivot to the right
    }
    arr[low] = pivot; // the pivot settles into its final position
    return low;
}

void QuickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot_pos = partition(arr, low, high);
        QuickSort(arr, low, pivot_pos - 1);
        QuickSort(arr, pivot_pos + 1, high);
    }
}
```